BeautifulSoup 사용법

1. 설치 및 환경 준비

BeautifulSoup 을 사용하기 위해서는 먼저 파이썬 환경에 라이브러리를 설치해야 합니다.

- **기본 설치:** 터미널에서 pip install beautifulsoup4 를 실행합니다.
- **협업 라이브러리:** 웹 페이지의 HTML 소스를 가져오기 위한 **requests** 라이브러리와 더 빠른 구문 분석을 위한 **lxml** 파서를 함께 설치하는 것이 권장됩니다 (pip install requests lxml).
- **가상 환경 권장:** 프로젝트 간 종속성 충돌을 방지하기 위해 가상 환경(venv)을 생성하여 격리된 공간에 설치하는 것이 모범 사례입니다.

```
(base) C:\Users\user>conda activate textmine26

(textmine26) C:\Users\user>pip install beautifulsoup4
Requirement already satisfied: beautifulsoup4 in .\anaconda3\envs\textmine26\lib\site-packages (4.13.5)
Requirement already satisfied: soupsieve>1.2 in .\anaconda3\envs\textmine26\lib\site-packages (from beautifulsoup4) (2.5
)
Requirement already satisfied: typing-extensions>=4.0.0 in .\anaconda3\envs\textmine26\lib\site-packages (from beautiful
soup4) (4.15.0)

(textmine26) C:\Users\user>
```

2. BeautifulSoup 객체 생성 (Parsing)

웹 크롤링의 첫 단계는 가져온 HTML 데이터를 파이썬이 이해할 수 있는 객체로 만드는 것입니다.

1. **데이터 요청:** requests.get(url)을 사용하여 페이지 콘텐츠를 가져옵니다. 이때 인코딩 문제를 방지하기 위해 res.content 를 사용하는 것이 좋습니다.
2. **객체 생성:** BeautifulSoup(markup, "파서이름") 형식을 사용합니다.
    - **html.parser:** 파이썬 내장 파서로 별도 설치가 필요 없으나 속도가 보통입니다.
    - **lxml:** 매우 빠르고 유연하며 HTML 과 XML 을 모두 지원하여 가장 권장됩니다.
    - **html5lib:** 웹 브라우저와 유사하게 가장 관대하게 파싱하지만 속도가 느립니다.

```python
import requests
from bs4 import BeautifulSoup

# 1. 대상 URL 설정 및 데이터 요청
url = "https://www.kopo.ac.kr/seongnam/board.do?menu=13152"
response = requests.get(url)

# 2. BeautifulSoup 객체 생성
soup = BeautifulSoup(response.content, "html.parser")

# 이제 'soup' 객체를 통해 데이터를 탐색할 수 있습니다.
print(soup.title.get_text())
```

```
문제   출력   디버그 콘솔   터미널   포트

PS D:\Lecture\TextMining26> cd "d:\Lecture\TextMining26\test_folder"
PS D:\Lecture\TextMining26\test_folder> C:\Users\user\anaconda3\envs\textmine26\python.exe -u beautifulSoup_test.py
학과게시판 > 인공지능소프트웨어 > 하이테크과정(대졸자과정) > (비학위)직업교육과정(무료직업교육) > 학과안내 | 성남캠퍼스
```

3. 주요 기능 및 사용법

탐색 및 검색 (Searching)

- **직접 접근:** soup.title, soup.a 와 같이 점(.)을 이용해 해당 태그의 첫 번째 요소를 즉시 가져올 수 있습니다.

```python
# 문서의 title 태그 내용 가져오기
print(soup.title.string)
# 문서의 첫 번째 span 태그 가져오기
print(soup.span.text)
```

```
문제   출력   디버그 콘솔   터미널   포트                    Code - test_fo

PS D:\Lecture\TextMining26\test_folder> cd "d:\Lecture\TextMining26\test_folder"
PS D:\Lecture\TextMining26\test_folder> C:\Users\user\anaconda3\envs\textmine26\python.exe -u tempCode
PS D:\Lecture\TextMining26\test_folder> cd "d:\Lecture\TextMining26\test_folder"
PS D:\Lecture\TextMining26\test_folder> C:\Users\user\anaconda3\envs\textmine26\python.exe -u beautifu
학과게시판 > 인공지능소프트웨어 > 하이테크과정(대졸자과정) > (비학위)직업교육과정(무료직업교육) > 학과
서울.경기 : 서울정수캠퍼스, 서울강서캠퍼스, 성남캠퍼스, 융합기술교육원, 인천캠퍼스, 남인천캠퍼스, 화성
육원, 반도체융합캠퍼스, 직업교육연구원
```

- **find() & find_all():**
  - find()는 조건에 맞는 **첫 번째** 요소 하나만 반환하며, 없으면 None 을 반환합니다.
  - find_all()은 조건에 맞는 **모든** 요소를 리스트 형태로 반환합니다.
  - 특정 속성을 지정하려면 attrs={"class": "클래스명"}과 같이 딕셔너리 형태로 인자를 전달합니다.

```python
# 태그 하나만 찾기
fBoard = soup.find('a', {"class":"nav-skip__item"})
print(fBoard)
# 태그 전부를 리스트로 가져오기
fBoards = soup.find_all('a', {"class":"nav-skip__item"})
print(fBoards)
```

```
문제   출력   디버그 콘솔   터미널   포트                    Code - test_folder

PS D:\Lecture\TextMining26\test_folder> cd "d:\Lecture\TextMining26\test_folder"
PS D:\Lecture\TextMining26\test_folder> C:\Users\user\anaconda3\envs\textmine26\python.exe -u beautifulSoup_test.py
<a class="nav-skip__item" href="#container">성남캠퍼스 본문 바로가기</a>
[<a class="nav-skip__item" href="#container">성남캠퍼스 본문 바로가기</a>, <a class="nav-skip__item" href="#kopoMenu">성남캠
퍼스 주메뉴 바로가기</a>]
```

- **select():** CSS 선택자를 사용하여 soup.css.select("p > a")처럼 더 복잡한 경로로 요소를 찾을 수 있습니다.

```python
# class가 b_subject안의 span태그 안의 a태그 찾기
tabledata = soup.select_one(".b_subject > span > a")
print(tabledata.text)

tdtext = soup.select(".b_subject > span > a")
for tdt in tdtext:
    print(tdt.text) # 태그 안의 속성이 아니라 메서드 사용
    print(tdt["href"])  # 태그안의 속성은 []로 접근 가능
```

- **관계 탐색:** 현재 요소의 부모(find_parent), 모든 부모(find_parents), 이후에 나오는 형제들(find_next_siblings) 등을 찾을 수 있습니다.

```python
# 2. 게시물 목록(행) 가져오기
# 공지사항 테이블의 모든 행(tr)을 가져옵니다.
rows = soup.select('table.tbl_list > tbody > tr')

for row in rows:
    # 3. 기준점 잡기: 제목이 들어있는 td를 먼저 찾습니다.
    # 클래스명이 'left'인 td가 제목을 포함하고 있습니다.
    title_td = row.find('td', class_='b_subject')

    if title_td:
        # 제목 텍스트 추출
        title = title_td.get_text(strip=True)

        # 4. 관계 탐색 시작 (형제 노드 찾기)
        # 제목(td) 바로 다음에 오는 형제 td들을 모두 가져옵니다.
        # [작성자td, 등록일td, 조회수td] 순서로 리스트가 반환됩니다.
        siblings = title_td.find_next_siblings('td')

        if len(siblings) >= 2:
            author = siblings[0].get_text(strip=True) # 첫 번째 형제: 작성자
            date = siblings[1].get_text(strip=True)   # 두 번째 형제: 등록일

            print(f"제목: {title}")
            print(f"작성자: {author} | 등록일: {date}")
            print("-" * 50)
```

데이터 추출 (Extraction)

- **텍스트 추출:**
  - get_text(): 태그와 하위 태그에 포함된 **모든 텍스트**를 하나의 문자열로 가져옵니다.
  - .string: 태그 내부에 **텍스트 노드가 하나만 있을 때** 그 내용을 반환하며, 자식 태그가 여럿이면 None 을 반환합니다.
- **속성 추출:** 태그 내의 href, src 같은 속성은 tag['href'] 또는 tag.get('href')를 통해 딕셔너리 접근 방식으로 가져옵니다.

구조 수정 (Modification)

BeautifulSoup 은 파싱된 트리를 직접 수정하는 기능도 제공합니다.

- **삭제:** extract()는 요소를 제거하고 반환하며, decompose()는 완전히 삭제합니다. clear()는 요소의 모든 자식을 제거합니다.
- **추가 및 교체:** append()나 extend()로 새 요소를 추가하거나, replace_with()로 기존 요소를 바꿀 수 있습니다.

4. 데이터 저장 및 문제 해결

- **JSON 저장:** 수집한 데이터를 딕셔너리에 담아 json.dumps(data, ensure_ascii=False)를 사용하면 한국어 깨짐 없이 파일로 저장할 수 있어 나중에 활용하기 편리합니다.
- **일반적인 문제:** ModuleNotFoundError 가 발생하면 활성화된 가상 환경에 bs4 가 설치되었는지 확인해야 하며, 파서 관련 오류 발생 시 해당 파서(예: lxml)가 설치되어 있는지 확인해야 합니다

Selenium 사용법

1. 요소(Element) 찾기

데이터를 추출하기 전, 가장 먼저 해당 데이터가 HTML 문서의 어디에 위치하는지 컴퓨터에게 알려줘야 합니다. find_element 메서드를 사용하며, 가장 권장되는 방식은 **By.CSS_SELECTOR** 와 **By.XPATH** 입니다.

- **ID/Class 활용:** (By.ID, "id_name"), (By.CLASS_NAME, "class_name")
- **CSS 선택자:** (By.CSS_SELECTOR, "div.content > ul > li")
- **XPath:** (By.XPATH, '//*[@id="main"]/div[1]') (경로가 복잡할 때 유용)

2. 텍스트 추출: .text

가장 기본이 되는 방법입니다. 해당 태그 사이에 적힌 글자를 가져옵니다.

```
# 1. 크롬 옵션 설정 (필요 시)
options = webdriver.ChromeOptions()
# options.add_argument('--headless') # 브라우저 창을 띄우지 않고 실행하고 싶을 때

# 2. 드라이버 자동 설치 및 서비스 객체 생성
service = Service(ChromeDriverManager().install())

# 3. 드라이버 실행
driver = webdriver.Chrome(service=service, options=options)

# --- 이제부터 데이터를 추출하면 됩니다 ---
driver.get("https://www.google.com")
print(driver.title)

# 작업 완료 후 닫기
driver.quit()
```

3. 속성값 추출: .get_attribute()

글자가 아니라 태그 안에 숨겨진 정보(링크 주소, 이미지 경로 등)를 가져오고 싶을 때 사용합니다.

- **링크(URL) 가져오기:** element.get_attribute("href")
- **이미지 경로 가져오기:** element.get_attribute("src")

- **입력창의 값 가져오기:** element.get_attribute("value")

```python
# 1. 해당 게시판 페이지로 이동
driver.get("https://www.kopo.ac.kr/seongnam/board.do?menu=13152")
time.sleep(2) # 페이지 로딩 대기

# 2. 첫 번째 게시물의 제목 요소 찾기
first_post = driver.find_element(By.CSS_SELECTOR, "td.b_subject a")

# 3. .text와 .get_attribute() 비교해보기
title_text = first_post.text  # 눈에 보이는 제목 글자
post_url = first_post.get_attribute("href")  # 숨겨진 연결 주소

print(f"제목: {title_text}")
print(f"링크: {post_url}")
```

```python
try:
    # 첨부파일 아이콘 이미지 요소 찾기
    file_icon = driver.find_element(By.CSS_SELECTOR, "img[src*='file']")

    # 이미지의 alt 속성(대체 텍스트) 가져오기
    file_desc = file_icon.get_attribute("alt")
    print(f"첨부파일 아이콘 설명: {file_desc}")
except:
    print("첨부파일이 있는 게시물이 현재 페이지에 없습니다.")
```

4. 여러 개의 데이터 한 번에 가져오기: find_elements

게시판의 목록이나 상품 리스트처럼 반복되는 구조를 가져올 때는 element 뒤에 's'를 붙인 find_elements 를 사용합니다. 이 메서드는 리스트(List) 형태로 결과값을 반환합니다.

```python
# 1. 모든 제목 요소를 '그물'로 건져 올립니다.
title_list = driver.find_elements(By.CSS_SELECTOR, "td.b_subject a")

# 2. 리스트 형태이므로 for문을 사용해 하나씩 꺼냅니다.
print(f"총 {len(title_list)}개의 게시물을 찾았습니다.")
print("-" * 30)

for title in title_list:
    # 각 요소(title)에서 텍스트와 링크를 추출합니다.
    name = title.text
    link = title.get_attribute("href")

    print(f"제목: {name}")
    print(f"링크: {link}")
    print("-" * 10)

driver.quit()
```

```python
# 모든 행(Row) 가져오기 (헤더 제외하고 본문 tr들만)
rows = driver.find_elements(By.CSS_SELECTOR, "tbody tr")

for row in rows:
    # 각 행 안에 있는 칸(td)들을 다시 찾습니다.
    # row.find_elements (driver가 아니라 row에서 찾음!)
    cols = row.find_elements(By.TAG_NAME, "td")
    if len(cols) > 1: # 데이터가 있는 행인지 확인
        subject = cols[1].text  # 제목 (성남폴리텍 게시판 기준 3번째 칸)
        date = cols[3].text     # 작성일 (성남폴리텍 게시판 기준 6번째 칸)

        print(f"{subject} | {date}")

driver.quit()
```